A 3-HOUR HANDS-ON TUTORIAL

Closing the Loop

Vibe coding with the **pi** agent — and the **pi-30** fleet benchmark

Install and validate pi on your machine · run 30 verifier-gated coding loops ·
read the results of 26 model × backend lanes — and learn why the zeros lie

install & validate | all 30 problems | harness engineering | war stories | fleet results

Rick Stevens · pi-30 fleet benchmark (v8) · July 2026

LEARNING OBJECTIVES

What you'll walk out with

A working pi install

pi 0.82 on your laptop, talking to a real model — cloud API, OpenRouter, or a fully local Ollama / LM Studio endpoint — validated end-to-end.

Loops you can trust

You'll run verifier-gated agent loops — iterate-until-green, oracle checks, measurement loops, tournaments — where exit codes, never prose, decide PASS.

Fleet-scale judgment

You'll read the 26-lane Fleet-30 results like an operator: spot infra failures masquerading as dumb models, and route real work to lanes that earn it.

Ground rule for the whole tutorial: a claim only counts if a machine-checkable verifier returned exit code 0. That applies to the models — and to us.

AGENDA

The 3-hour flightplan

Two labs, one break, six parts — timings are on every section divider so you always know where we are.

0:00	**Welcome**	Why agent loops — and why leaderboards can't see them
0:10	**Part 1 · Meet pi**	The minimal agent: install, validate, providers, sessions
0:35	**LAB 1**	pi running on YOUR machine — first verified loop
0:50	**Part 2 · Benchmark design**	Six standing rules, the repo, the L1–L10 method ladder
1:15	**Part 3 · The 30 problems**	All five families — every task, every trap
1:45	**Break**	10 minutes
1:55	**Part 4 · Harness engineering**	Timeouts, path clamps, resume, 13-lane fleet mode
2:15	**LAB 2**	Run a real lane against your model
2:30	**Part 5 · War stories**	The zero-score files: five infrastructure whodunits
2:45	**Part 6 · Results**	26 lanes, 4 charts, 5 durable findings
2:58	**Close**	Author P31, resources, Q&A

BEFORE WE START

Setup checklist — get these while we talk

Everything below is free. If anything is missing, Lab 1 has catch-up time built in.

A laptop shell

macOS, Linux, or Windows + WSL2. You'll live in the terminal today.

Node.js ≥ 20

node --version — pi is an npm package. nvm makes this painless.

Python 3.10+ · numpy · pytest

The verifiers run on plain CPython: pip install numpy pytest.

One model path

An API key (Anthropic / OpenAI / Google / OpenRouter) — or a local Ollama / LM Studio model. Both paths covered in Part 1.

git + coreutils

git clone for the repo; gtimeout (brew install coreutils on macOS) — the harness refuses to run without it.

The repo

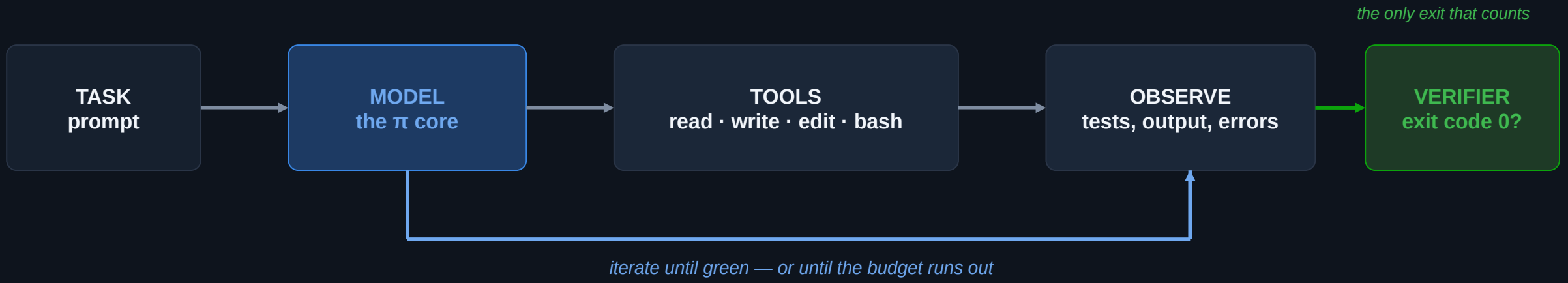
github.com/rick-stevens-ai/pi-30 — problems, harness, runs, report, this deck.

Zero-hardware track: no GPU required today. OpenRouter's free tier + the harness's code paths work from any laptop — local GPUs and clusters are a bonus, not a prerequisite.

THE BIG IDEA

Models aren't chatbots here — they're loop cores

An agent loop: the model reads a task, acts through tools, observes reality, and iterates until an independent judge says done.



file-path discipline

artifacts must land where the grader looks

recover from red

read a pytest traceback, make the smallest fix

drive multi-step tools

no wandering, no giving up early

know when it's done

stop on green — don't 'improve' passing code

THE MEASUREMENT GAP

Why the usual leaderboards can't see this

What standard evals measure

- MMLU / GSM8K — recall & reasoning in isolation
- Chat-arena Elo — how answers feel to humans
- One-shot codegen — no tools, no feedback, no state
- **None of them close a loop.**

What Fleet-30 measures

- Can the model drive pi's tools to a verified fix?
- Per problem: binary PASS/FAIL from a grader's exit code
- Per lane: model × backend — the serving stack is part of the product
- **30 loops closed = one number you can route work on.**

> *"A model can be fluent in isolation and still be useless in a loop if it cannot follow a file-path discipline, recover from a failing test, or drive a multi-step tool sequence without wandering."* — Fleet-30 report, §1

OPERATING PHILOSOPHY

“Vibe coding”, the disciplined version

Delegating code to an agent loop is a vibe. Grading it is a science. We do both, in that order.

Delegate the loop, not the keystrokes

You author the task, the constraints, and the grader. pi drives the edit-run-read cycle. Your leverage is problem design, not typing speed.

Trust only the verifier

“All tests pass! 🎉” from a model is not evidence. A verifier's exit code is. Every verdict in this tutorial — including our own demos — comes from code we don't let the model edit.

Instrument everything

RESULTS.txt per lane, heartbeats per problem, logs per call, archived BROKEN-* runs. When (not if) something breaks, provenance is what saves the week.

Motto of the day: verdicts from exit codes, never vibes.

THE CAST

One agent, thirty problems, a whole fleet

30	10	26	5	8
verifier-gated problems	loop methods (L1–L10)	model × backend lanes	serving paths	clean 30/30 lanes

Three problem sets live in this program — segmented, never cross-merged:

Fleet-30

P1–P30 · this repo · run_model_30.sh — today's main stage

aggB-10

P1–P10 canonical · a second operator's independent harness

mesh-extract-10

10-problem subset · sweeps on the spark mesh

⚠ **Provenance trap:** the three sets number their problems differently. Merging their matrices mis-segments columns — it happened once, it's now a standing rule. (§0 of the master report.)

PART 1 · 0:10 – 0:35 · ~25 MIN

Meet pi

A minimal, MIT-licensed agent harness: unified LLM API, an agent loop, four tools, and ferocious extensibility. We install it, validate it, and wire it to your models.

what & why | install + validate | providers & models.json | sessions & steering

THE AGENT

pi: a coding agent that fits in your head

- A terminal coding agent + agent toolkit — “minimal agent harness” is the official self-description.
- Born as badlogic/pi-mono (Mario Zechner); now developed as earendil-works/pi — “Earendil Pi” in the Fleet-30 report.
- MIT-licensed, open source, distributed on npm as @earendil-works/pi-coding-agent.
- Four built-in tools only: read · write · edit · bash. Everything else is an extension you opt into.
- 15+ providers out of the box (Anthropic, OpenAI, Google, Groq, Bedrock…) — plus any OpenAI-compatible endpoint via models.json. That last part powers this whole benchmark.

SITE / DOCS

`pi.dev · pi.dev/docs/latest`

SOURCE

`github.com/earendil-works/pi`

INSTALL

`npm i -g @earendil-works/…`

VERSION TODAY

`0.82.0 (verified 2026-07-24)`

LICENSE

`MIT`

MODES

`TUI · print · JSON · RPC/SDK`

Why pi for a benchmark? a tiny, inspectable harness means the variable under test is the model × serving stack — not a labyrinth of agent scaffolding.

DESIGN PHILOSOPHY

Minimal core, deliberate omissions

pi ships without features most agents brag about — on purpose. Each omission is an extension point instead.

- ~~built-in MCP~~
- ~~sub-agents~~
- ~~permission popups~~
- ~~plan mode~~
- ~~to-do lists~~
- ~~background bash~~

…all installable as TypeScript extensions, skills, or packages when you want them — none taxing the core loop when you don't.

What the minimal core buys you

- A short, honest system prompt — model behavior stays legible
- Auto-compaction + dynamic context injection built in
- Extensions: custom tools, commands, UI; skills load on demand
- AGENTS.md / SYSTEM.md project context files

What it does NOT buy you

- No built-in permission system: pi will read, write, and execute whatever your shell can.
- Official guidance: containerize for isolation — Docker, the Gondolin extension, or OpenShell.
- The Fleet-30 harness runs it inside per-problem scratch dirs + timeouts for exactly this reason.

ARCHITECTURE

One monorepo, four packages

Each layer is useful alone — the coding agent is just the top of the stack.



For tinkerers: npm install --ignore-scripts && npm run build in the monorepo, then ./pi-test.sh runs pi from source in any directory — handy for hacking on extensions.

THE LOOP, INSTANTIATED

pi's working parts: tools + context

read

inspect files before opining

write

create artifacts (graded later!)

edit

surgical diffs, smallest fix

bash

run pytest, benchmarks, anything

Context files pi reads on startup

- AGENTS.md — project instructions, discovered from cwd, parents, and ~/.pi/agent/
- SYSTEM.md — override or extend the system prompt per project
- memory.md — agent memory; two Fleet-30 problems grade whether the model appends lessons to it

ANATOMY OF ONE LOOP

```
# a pi turn, seen from the transcript
user  fix the failing test in parens.py
tool  bash(pytest -q test_thing.py)
      1 failed, 6 passed
tool  read(parens.py)
tool  edit(parens.py: stack.pop fix)
tool  bash(pytest -q test_thing.py)
      7 passed  ← the loop closed
```

FOUR WAYS TO RUN IT

TUI for humans, print/JSON/RPC for harnesses

1 · INTERACTIVE TUI

```
$ pi
# interactive TUI: steer live, Ctrl+L to
# switch models, Enter interrupts a run
$ pi "Summarize this repo"
```

2 · PRINT MODE (the harness's workhorse)

```
$ pi -p "List all TODOs in src/"
# one shot: run tools, print, exit 0
# stdin works too:
$ cat log.txt | pi -p "root cause?"
```

3 · JSON EVENTS

```
$ pi -p --mode json "..."
# newline-delimited event stream:
{"type":"tool_call","name":"bash",…}
{"type":"message","content":…}
```

4 · RPC / SDK

```
// SDK: same loop, embedded
import { createAgent } from
  "@earendil-works/pi-coding-agent"
// or --mode rpc for editor/IDE bridges
```

INSTALL

Two clean paths onto your machine

PATH A · NPM (recommended)

```
# with Node ≥ 20 already installed
$ npm install -g --ignore-scripts \
    @earendil-works/pi-coding-agent

# --ignore-scripts: pi needs no install
# hooks; refusing lifecycle scripts cuts
# supply-chain risk. Recommended by docs.
```

PATH B · ONE-LINER

```
$ curl -fsSL https://pi.dev/install.sh \
    | sh

# installer script for macOS / Linux;
# pnpm and bun also supported

$ pi update      # upgrades later
```

macOS · Linux · Windows (WSL2 / Termux docs exist)



Supply-chain posture: pi pins exact versions for all direct dependencies, treats package-lock.json as authoritative, and runs audit + signature checks in CI. --ignore-scripts is belt-and-suspenders on top.

VALIDATE

Prove the install before trusting it

REAL OUTPUT · CLEAN LINUX SANDBOX · 2026-07-24

```
$ pi --version
0.82.0

$ which pi
~/.npm-global/bin/pi

$ pi --help | head -2
pi - AI coding assistant with read,
     bash, edit, write tools

$ pi --list-models | head -3
provider   model           context  max-out
bedrock    amazon.nova-2…  128K     4.1K
anthropic  claude-…        200K     64K
```

The 3-step validation ritual

- 1 · Binary answers: --version returns without error → Node + PATH are right.
- 2 · Catalog loads: --list-models prints the built-in provider catalog → install is complete (works before any API key).
- 3 · A real loop closes: after auth (next slide), pi -p "create hello.txt containing hi" && cat hello.txt → the file exists. Tools verified.

sandbox-verified

AUTHENTICATE

Three ways to give pi a brain

A · Subscription login

```
$ pi
> /login
# OAuth: Claude Pro/Max,
# ChatGPT, GitHub Copilot
```

B · API key via env

```
$ export ANTHROPIC_API_KEY=sk-…
$ export OPENROUTER_API_KEY=…
$ pi
# picked up automatically
```

C · Stored key

```
> /login  # pick api-key
# saved to
~/.pi/agent/auth.json
# survives new shells
```

Which one today? For the labs: B with OpenRouter (free tier works, throttled) or a local endpoint with a dummy key. The fleet's standing policy: free/sanctioned endpoints, and every score is attributed to provider × model.

CUSTOM PROVIDERS

models.json — the benchmark's secret door

Any OpenAI-compatible endpoint becomes a first-class pi provider. This is how 26 lanes across 5 serving paths all ran the same harness.

~/.pi/agent/models.json

```
// ~/.pi/agent/models.json
{ "providers": {
  "ollama": {
    "baseUrl": "http://localhost:11434/v1",
    "api": "openai-completions",
    "apiKey": "ollama",
    "models": [{
      "id": "qwen3-coder:30b",
      "contextWindow": 128000,
      "maxTokens": 32000
    }]
} } }

$ pi --provider ollama --model qwen3-coder:30b
```

Field notes

- api: openai-completions · openai-responses · anthropic-messages · google-generative-ai
- apiKey accepts literals, $ENV_VAR interpolation, or !cmd shell-outs — keys stay out of the file
- maxTokens = client-side output budget; the server's context length is separate (a distinction that debugged a 0/30 — Part 5)
- Hot-reloads every time /model opens — no restart while tuning a fleet

CUSTOM PROVIDERS, IN ANGER

The fleet's actual provider roster

Names from the real ~/.pi/agent/models.json that ran the benchmark — one name = one serving path, and lanes are attributed to it.

spark36	`http://<spark-node>:11434/v1`	Ollama on a DGX Spark node — no auth, local net
lmstudio-m3	`http://<mac-studio>:1234/v1`	LM Studio on a Mac — JIT model loading, 55 models
openrouter-free	`https://openrouter.ai/api/v1`	hosted free tier — 429-throttled; key via env
chiatta00-agg	`http://127.0.0.1:8000 (SSH tunnel)`	LiteLLM aggregator fronting 13 models on 2 GPU hosts
cels-direct	`http://<dgx-endpoint>/v1`	local DGX vLLM endpoints — swept direct, never proxied

SESSIONS

Branching history you can keep — or refuse

SESSION CONTROL

```
$ pi --name "refactor auth"   # named
$ pi -c            # continue most recent
$ pi -r            # browse & resume any
$ pi --fork 3f2a   # branch a session
$ pi --export run.html # share as HTML

# and the harness's favorite:
$ pi --no-session -p "…"
# ephemeral: nothing written, no state
# leaks between 30 problems × N rounds
```

Session trees

- Every message is a node — branch from any point in history and explore alternatives
- Export to standalone HTML or share via GitHub gist
- Stored locally per project; --session-dir relocates them



Benchmark discipline: lanes run --no-session. Determinism beats convenience when 26 fleets share one machine.

LIVE STEERING

Driving the TUI like you mean it

`Enter`	interrupt the running agent — mid-tool-call steering
`Alt+Enter`	queue a follow-up without interrupting
`Ctrl+L`	switch model mid-session (state carries over)
`Ctrl+P`	cycle favorite models (--models "glob,patterns")
`! cmd`	run a shell command yourself, output lands in context
`@file`	attach files or images to the next message
`/model  /login  /export`	slash commands — extensions can add more

CHEAT SHEET

The seven flags the whole benchmark rides on

Memorize these and you can read every line of run_model_30.sh.

flag	what it does	why the harness needs it
`-p / --print`	one-shot: run tools, print, exit	every single harness call
`--provider X --model Y`	pin the lane identity	score attribution = provider × model
`--no-session`	ephemeral run, no files written	no state leaks across 30 problems
`-na / --no-approve`	ignore project-local config	a problem dir can't inject extensions
`--append-system-prompt T`	add text/file to system prompt	carries the PATH_CLAMP (Part 4)
`--thinking L`	reasoning budget: off…max	reasoning models (ornith, glm)
`--no-tools`	disable all tools	extraction-fallback harness only

LAB

LAB 1 · 0:35 – 0:50 · 15 MIN

First loop on your laptop

Install pi, validate it, authenticate, and close one real agent loop — a file on disk that a command you ran can verify.

install · validate · authenticate · one verified loop

Timebox 15 minutes. Helpers are roaming. If you finish early, try the stretch goal on the next slide — or help a neighbor (it counts as a tool call).

LAB 1 · WORKSHEET

Four steps, four checkpoints

1 · Install

```
$ npm install -g --ignore-scripts \
   @earendil-works/pi-coding-agent
# or: curl -fsSL pi.dev/install.sh | sh
```

✓ checkpoint: pi --version prints 0.8x.y

2 · Validate

```
$ pi --version && pi --list-models | head
# catalog prints with NO key configured
```

✓ checkpoint: a provider/model table appears

3 · Authenticate

```
$ export OPENROUTER_API_KEY=sk-or-…
# or /login inside pi · or local endpoint
```

✓ checkpoint: pi -p "say READY" answers

4 · Close a loop

```
$ mkdir lab1 && cd lab1
$ pi -p "write hello.py that prints 6*7;
   then run it"
$ python3 hello.py    # ← YOU verify
```

✓ checkpoint: 42 — from a file pi wrote

Stretch goal: pi --export lab1.html — open your session transcript in a browser and find the tool_calls.

No API key? Serve a model from your own metal

OPTION 1 · OLLAMA

```
$ ollama pull qwen3-coder:30b
$ ollama serve   # :11434

# register in ~/.pi/agent/models.json
# (stanza from slide 18), then:
$ pi --provider ollama \
     --model qwen3-coder:30b

⚠️ tool-call parsing varies per model
#   — Part 4 has the full story
```

OPTION 2 · LM STUDIO

```
# LM Studio: load a model, then
$ lms server start --bind 0.0.0.0

# verify it listens beyond localhost:
$ lsof -nP -iTCP:1234 -sTCP:LISTEN
  *:1234     ✓ good
  127.0.0.1:1234  ✗ localhost-only

# reload model after restart:
$ lms load <model> --ttl 3600
```

⚠ **Fleet scar tissue #1:** every early “connection error” against LM Studio was the server bound to 127.0.0.1 — instant resets for any remote peer, while local CLI calls worked fine. And the default context is 4096 tokens: bump it per-model or full agentic turns get truncated.

When it doesn't work (it's usually one of these)

symptom	usual cause	fix
`pi: command not found`	npm global bin not on PATH	npm config get prefix → add <prefix>/bin to PATH
`engine error on install`	Node < 20	nvm install 20 && nvm use 20, reinstall
`429 / empty replies`	free-tier throttling	retry with backoff; the fleet ships or_backoff_proxy.py for OpenRouter
`HTTP 000 in ~13 ms`	endpoint bound to localhost	serve with --bind 0.0.0.0 (LM Studio) / check firewall
`replies truncated mid-tool`	context window too small (4096)	raise per-model context; set maxTokens in models.json
`model talks, writes no files`	runtime lacks a tool-call parser	LM Studio or vLLM w/ --tool-call-parser; full matrix in Part 4



Rule of thumb: if pi answers in the TUI but never creates files, suspect the serving stack's tool-call parsing before you suspect the model. (Foreshadowing: that's Part 5's entire plot.)

PART 2 · 0:50 – 1:15 · ~25 MIN

Designing a benchmark you can actually trust

Six standing rules turn “the model seemed smart” into a number you can route production work on. Then: the repo, and the ten-rung method ladder.

- 6 standing rules
- repo tour
- method ladder L1–L10
- anatomy of a problem

02

THE BENCHMARK

pi-30 in one slide

1 problem = seed code (pre-verified broken) + task prompt + loop budget + **verifier (exit code = verdict)**

30 problems

P1–P30, authored as a curriculum: write-from-spec → fix-failing-tests → numerical → optimize → tournaments & critics.

1 harness

run_model_30.sh — problem statements embedded, every pi call timeout-wrapped, resume-safe, fleet-parallelizable.

1 lane = model × backend

The same weights on two serving stacks are two lanes. 26 unique lanes have completed the sweep.

1 verdict per problem

Binary PASS/FAIL from an independent grader the model may not edit. Score = passes out of 30. No partial credit, no vibes.

STANDING RULE 1 / 6

The verifier is the only source of truth

- Model prose is never trusted for scoring. Not “done!”, not “all tests pass”, not a beautiful summary.
- Each problem ships an independent grader: verify.py, check.py, pytest, a benchmark scorer, or a convergence check.
- The grader's process exit code is the verdict: 0 = PASS, anything else = FAIL.
- The model is forbidden from editing the grader — every prompt says so, and staging restores pristine copies each run.

run_model_30.sh + p1/verify.py

```
# harness verdict line — the whole religion:
(cd "$ROOT/p1" && "$PY" verify.py) \
  >/dev/null 2>&1 \
  && record "P1: PASS" \
  || record "P1: FAIL"

# verify.py ends with hard asserts:
assert rc == 0
assert abs(d["stdev"] - 2.138089935) < 1e-6
print("OK P1")   # exit 0 = the verdict
```

Why so strict? agent models are trained to sound finished. An eval that reads prose measures persuasion; an eval that runs code measures capability. (You'll see models claim victory over code that doesn't import.)

STANDING RULE 2 / 6

Seeds must fail first — the loop earns the fix

- Every fix/optimize problem starts from seed code that is pre-confirmed broken (or correct-but-slow for measurement loops).
- If the seed passed, a lane could score without doing anything — the benchmark would measure staging, not capability.
- Staging copies fresh seeds + verifiers for all 30 problems at lane start (rm -rf $ROOT first: never grade a prior run's artifacts).
- Demo → we ran P24's verifier against its untouched seed in a clean sandbox. It fails, loudly, as designed:

SEED-MUST-FAIL, DEMONSTRATED

```
# p24 seed — deliberately naive:
class RateLimiter:
    def allow(self):
        return True   # no limiting at all

$ python3 verify.py   # against the seed
BURST: allowed 10 at t=0, expected exactly 5
$ echo $?
1        ← seed fails ✓ (sandbox, 2026-07-24)
```

Corollary: a PASS is meaningful precisely because the starting state was a verified FAIL. Delta, not absolute state, is what's graded.

STANDING RULE 3 / 6

Tools always ON — the point is the loop

> *“No --no-tools runs. The point is the loop, not the model's zero-shot recall. A two-layer path clamp keeps the model working inside its problem directory; verdicts still come only from the verifier.”* — report §1

What tools-ON tests

Schema-correct tool_calls, path discipline, run-your-own-tests behavior, multi-step sequencing — the production skills.

What tools-OFF would test

One-shot code recall — a different (easier) exam. Kept only as a diagnostic fallback harness, clearly labeled, never merged.

The cost of the choice

Tool-use exposes the serving stack: parsers, templates, streaming. That's a feature — those bugs bite production too.

STANDING RULE 4 / 6

Single-shot is canonical; “effective” is a footnote

Case study: laguna-xs2 (33B-active MoE, single A100)

27/30
canonical single-shot
(missed P7 · P17 · P25)

→

3/3
targeted rerun of exactly
the three misses — all PASS

→

30/30
“effective” — reported, but
never promoted to headline

Why not promote it? Re-rolling only the misses counts the upside of variance but never re-checks whether a passing problem would drop on a second roll. The honest path to a true 30/30 claim is a full k-of-N sweep — same method for every lane. (A fresh cold pass later did hit 30/30 as uic-laguna-xs2, confirming the ceiling honestly.)

STANDING RULE 5 / 6 — THE MARQUEE

The Zero-Score Rule

Any 0/N lane MUST be treated as a failure of the server / harness / stack — NOT the model. Debug it as infrastructure, mark it BROKEN-<cause>, and never publish a literal zero as a capability result.

Standing policy, 2026-07-04 · applies to ALL benchmarks

`oss120`	0/30	vLLM 0.13 streaming-flush bug	30/30
`uic-ornith-9b`	0/30	Ollama served no chat template → no tool_calls	28/30
`nemotron-super`	0/27	phantom — model never resident on the host	26/30
`glm-4.7-flash`	0/23	2 real harness bugs — fixed, then re-run	1/28 (real ceiling)

Every zero in this program was root-caused. Three were pure infrastructure; one — glm — survived the audit as a genuine capability floor. Full autopsies in Part 5.

STANDING RULE 6 / 6

Attribute provider × model — never the model alone

Same gpt-oss-120b weights. Same harness. Same month.



Same story in wall-time

- 9 min (DGX) vs 63 min (free tier) for the full 30
- measurement problems ~800× slower on the degraded path
- the free endpoint effectively serves a throttled, truncated variant

Therefore: a lane is a (model × backend) pair, and the same model on two backends is two rows of the master table. “How good is model X?” is under-specified — ask “how good is X, served like this?”

THE ARTIFACT

Repo tour: rick-stevens-ai/pi-30

git clone github.com/rick-stevens-ai/pi-30

```
pi-30/
├─ problems/  p1/ … p30/
│    verifiers + seed scaffolding
├─ harness/
│    run_model_30.sh  ← prompts live HERE
│    fleet_*.sh  sweep_*.sh
│    or_backoff_proxy.py  ENGINEERING_NOTES.md
├─ runs/       one dir per lane + RESULTS.txt
├─ final_results/  frozen snapshots
├─ report/     LaTeX + PDF (v8)
├─ deck/       pptxgenjs slide generator
└─ FLEET_BENCHMARK_MASTER.md  ← start here
```

Three details people miss

- Problem statements are IN the harness, not in problems/ — one prompt block per problem inside run_model_30.sh. The problem dirs hold only graders + seeds.
- runs/ preserves failures: _BROKEN-* dirs are archived with self-documenting names, never deleted. Provenance is a first-class artifact.
- Everything regenerates: the report tables and the official deck are built from the same RESULTS.txt files you'll produce in Lab 2.

THE CURRICULUM

The method ladder: ten ways to run a loop

Each rung adds one agentic skill. The 30 problems instantiate the rungs — same ladder, rising stakes.

L1
single-shot
one call, then grade

L2
bundled tests
write + RUN your own tests

L3
iterate-until-green
pytest fails fed back, N rounds

L4
oracle check
diagnostic diff vs reference

L5
measurement
optimize to a throughput target

L6
generator + critic
hostile self-review gate

L7
fan-out
parallel sub-builds + assembly

L8
reflection
fix + write a lesson to memory

L9
tournament
best-of-N candidates scored

L10
capstone
tournament + critic + bench

difficulty & agency →

THE FULL MAP

All 30 problems on the ladder

L1 single-shot	P1	stats CLI from spec
L2 bundled tests	P2	LRU cache + write & run pytest
L3 iterate-until-green	P3 P11 P13 P20 P22 P29	roman · parens · mini-JSON · TTL cache · toposort · calculator
L4 oracle check	P4 P12 P14 P21 P28	softmax · base-N · Dijkstra · intervals · determinant
L5 measurement	P5 P15 P23	matmul GFLOP/s · wordcount Mtok/s · primes Mn/s
L6 generator + critic	P6 P16 P24	FP reduce · thread-safe counter · token bucket
L7 fan-out	P7 P17 P25	parse backends · codec round-trip · 3-stage pipeline
L8 reflection	P8 P18 P26	RK4 ODE · Welford variance · Newton sqrt (+ memory.md)
L9 tournament	P9 P19 P27	levenshtein · fast-doubling fib · overlap count
L10 capstone	P10 P30	prime sieve · sort kernel (tournament + critic + bench)

ANATOMY

Inside one problem directory

problems/p24 (token bucket)

```
problems/p24/
├─ limiter.py      ← SEED (broken on
│                     purpose; must fail)
└─ verify.py       ← GRADER (never
                     edited by the model)

# graders across the suite:
verify.py          behavior asserts
check.py           oracle diff
test_thing.py      pytest suites
bench.py + check.py  speed + correctness
score.py           tournament scorer
converge_check.py  numerical convergence
PLAN.md / CRITIQUE.md  fan-out & critic aids
```

The staging contract (per lane)

- rm -rf runs30/<tag> — a lane never sees a previous lane's artifacts
- Fresh copies of every verifier + seed staged into runs30/<tag>/pN/
- pi runs with cwd = the problem dir; the grader runs from the same dir moments later
- Verifiers execute under a pinned Python (numpy + pytest available) — not whatever the model's bash found on PATH
- “DO NOT edit check.py/verify.py” appears in every prompt — and staging would clobber the edit anyway

PART 3 · 1:15 – 1:45 · ~30 MIN

Thirty problems, five families

Every task, every grader, every trap — with real fleet pass rates. Small enough to reason about, sharp enough to separate a 9B star from a 550B stumble.

A · write 5 | B · fix 6 | C · numerical 9 | D · optimize 2 | E · tournament 8

03 π

OVERVIEW

The five families at a glance

A

Write-from-spec

P1 P2 P7 P17 P25

spec → working code in an empty dir; run the verifier yourself

77%

B

Fix failing tests

P3 P11 P13 P20 P22 P29

red pytest in, smallest fix out — tests are untouchable

88%

C

Numerical

P4 P8 P12 P14 P18 P21 P23 P26 P28

the bug is mathematical: overflow, cancellation, wrong recurrence

88%

D

Optimize

P5 P15

correct-but-slow seed; hit a measured throughput target

100%

E

Tournaments & critics

P6 P9 P10 P16 P19 P24 P27 P30

best-of-N arenas + hostile self-review — the most agentic family

85%

mean fleet pass rate per family →

FAMILY A · WRITE-FROM-SPEC · 5 PROBLEMS

Family A — hand it a spec and an empty directory

P1 — 91%

stats CLI

STDIN numbers → one exact output line. Sample stdev, median rules, stdlib only.

P2 — 96%

LRU cache + own tests

O(1) get/put, eviction rules — AND write + run pytest before declaring done.

P7 — 74%

dispatch router

3 backends built in parallel; then a dispatcher that imports across dirs.

P17 — 61%

codec round-trip

dumps/loads × 3 formats, then round_trip(fmt,obj) through the right backend.

P25 — 61%

3-stage pipeline

clean → agg → fmt stages composed by pipeline.run(rows). Even top lanes slip.

What this family measures Spec-following + multi-file scaffolding + cross-directory imports. The two compose-what-you-built problems (P17, P25) are top-5 hardest in the whole suite — assembly is harder than parts.

FAMILY A · WRITE-FROM-SPEC

P1 — stats CLI

91%

fleet pass rate

L1 · single-shot

THE TASK (verbatim prompt)

> *"Write cli.py: reads whitespace/newline-separated numbers from STDIN, prints exactly one line count= min= max= mean= median= stdev= (stdev = SAMPLE stdev, n-1). median of even count = avg of two middle. Stdlib only. Exit 0."*

THE LOOP

One pi call, zero retries — then verify.py runs the CLI as a subprocess on known data and asserts every field to 1e-6.

The trap: sample vs population stdev. The verifier's magic number 2.138089935 only matches n−1. Missed only by the two smallest gemma lanes.

p1/verify.py (grader) + sandbox check

```python
def run(stdin):
    p = subprocess.run([sys.executable,
        "cli.py"], input=stdin, …)
    return p.stdout.strip(), p.returncode

data = [2, 4, 4, 4, 5, 5, 7, 9]
assert int(d["count"]) == 8
assert abs(d["median"] - 4.5) < 1e-9
assert abs(d["stdev"] - 2.138089935) < 1e-6
print("OK P1")

# our 7-line solve, clean sandbox:
$ python3 verify.py
OK P1            # exit 0  ✓
```

FAMILY A · WRITE-FROM-SPEC

P2 — LRU cache — write your own tests first

96%

fleet pass rate

L2 · bundled tests

THE TASK (verbatim prompt)

> *"Implement lru.py with class LRUCache(capacity): get(key)→value-or-None, put(key,value), O(1), correct LRU eviction (get and put both count as use; put on existing key updates value AND refreshes recency). ALSO write test_lru.py with pytest tests and RUN pytest before done."*

THE LOOP

Single call again — but the prompt demands the model author and execute its own test suite before declaring victory. The independent verifier then re-checks eviction order.

What it really grades: the test-before-done habit. 96% — only the 33B Nemotron-H Omni lane missed it fleet-wide.

the canonical winning shape

```python
from collections import OrderedDict

class LRUCache:
    def get(self, key):
        if key not in self.d: return None
        self.d.move_to_end(key)  # use!
        return self.d[key]
    def put(self, key, value):
        self.d[key] = value
        self.d.move_to_end(key)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

```
$ pytest -q test_lru.py   …all green
```

FAMILY A · WRITE-FROM-SPEC

P7 — parallel backends fan-out

74%

fleet pass rate

L7 · fan-out

THE TASK (verbatim prompt)

Three separate pi calls — each in its own work/<be>/ dir: "Per ../../PLAN.md implement ONLY the '<be>' backend in backend.py exposing parse(text)→dict." Then a fourth call: "Write dispatch.py routing kind ∈ {csv,kv,json}. Run verify.py."

THE LOOP

The harness itself is the orchestrator: 3 concurrent subshell pi calls (&), wait, then one assembly call. The model never sees its siblings.

Why 26% fail: the assembly step. Backends come out fine; the dispatcher's cross-directory import (work/csv/backend.py) is where small models and Nemotron-H lanes fumble.

run_model_30.sh · the fan-out block (verbatim)

```
for be in csv kv json; do
  ( mkdir -p "$ROOT/p7/work/$be" &&
    cd "$ROOT/p7/work/$be" &&
    $PT -p $PROV $NS "…ONLY the '$be'
      backend…" ) >/dev/null 2>&1 &
done
wait            # ← barrier: all 3 land

run_pi "$ROOT/p7" "Backends exist at
  work/{csv,kv,json}/backend.py … Write
  dispatch.py … DO NOT edit verify.py"
(cd "$ROOT/p7" && "$PY" verify.py)
  && record "P7: PASS" || record "P7: FAIL"
```

FAMILY A · WRITE-FROM-SPEC

P17 — serializer codec round-trip

61%

fleet pass rate

L7 · fan-out

THE TASK (verbatim prompt)

“Per ../../PLAN.md implement ONLY the '<fmt>' backend exposing dumps(obj)→str and loads(s)→obj” — for csv, kv, and pylit (repr / ast.literal_eval, never eval). Then: “Write codec.py exposing round_trip(fmt,obj) = loads(dumps(obj)) using the matching backend.”

THE LOOP

Same 3-parallel-then-assemble shape as P7 — now each backend carries TWO functions and the verifier asserts a round-trip invariant through each.

Top-4 hardest (61%). P7 with the contract doubled: nine lanes failed, all smaller/slower models. Holding a 2-function × 3-target contract without touching the verifier is the skill.

the contract that breaks the mid-tier

```
# work/pylit/backend.py — the sneaky one:
import ast
def dumps(obj): return repr(obj)
def loads(s):
    return ast.literal_eval(s)  # NOT eval

# codec.py — the graded assembly:
def round_trip(fmt, obj):
    be = _load_backend(fmt)   # work/<fmt>/
    return be.loads(be.dumps(obj))

# verifier: round_trip('pylit', {'a':[1,2]})
#           must equal the original object
```

FAMILY A · WRITE-FROM-SPEC

P25 — 3-stage stats pipeline

61%

fleet pass rate

L7 · fan-out

THE TASK (verbatim prompt)

Stages built in parallel: clean(rows)→floats (drop junk) · agg(nums)→{count,sum,mean,min,max} · fmt(d)→one line, keys alpha-sorted. Then: "Write pipeline.py exposing run(rows) = fmt(agg(clean(rows))) (import each stage). Run verify.py."

THE LOOP

3 parallel stage builds → 1 composition call → verifier feeds dirty rows ([None, '', 'x', '3.5', 7]) through run() and string-compares the formatted line.

The humbling one: the ONLY problem glm-5.2 (29/30) missed, and one of laguna's three single-shot misses. Function composition across an import tree has irreducible variance even at the top.

what the verifier feeds it

```
rows = ["3", None, "", "x", "4.5", 7]

clean(rows)   → [3.0, 4.5, 7.0]
agg(…)        → {count:3, sum:14.5,
                 mean:4.833…, min:3, max:7}
fmt(…)        → "count=3 max=7.0 mean=…"
                  # keys ALPHA-sorted!

# pipeline.py — three imports, one line:
def run(rows):
    return fmt(agg(clean(rows)))
```

FAMILY B · FIX FAILING TESTS · 6 PROBLEMS

Family B — red tests in, smallest fix out

A broken file + failing pytest/diagnostic. The model may not touch the tests. Restraint is the skill.

P3	91%	P11	91%	P13	70%	P20	91%	P22	96%
roman numerals		**balanced parens**		**mini-JSON parser**		**TTL-LRU cache**		**toposort**	
subtractive notation (IV, IX, XL…). The classic smallest-fix drill, 8 rounds max.		stack-match ()[]{} while ignoring every other character.		recursive descent over a JSON subset — json module and eval both banned.		LRU eviction AND expiry via an injected clock() — no real time allowed.		valid topological order; raise on cycles. Only llama70 missed.	

What this family measures Feedback-driven repair: each round pipes the tail of the failing pytest output into the next prompt. Strong lanes go green in 2; weak ones thrash sideways, 'fixing' code that was already right.

FAMILY B · FIX FAILING TESTS

P3 — the iterate-until-green loop itself

91%

fleet pass rate

L3 · iterate-until-green

THE TASK (verbatim prompt)

“pytest test_thing.py failing. Fix roman.py to use subtractive notation (IV IX XL XC CD CM). Smallest fix. DO NOT edit test_thing.py. Failures: <last 25 lines of pytest>”

THE LOOP

Up to 8 rounds. Each round: run pytest → green? PASS “in i”. Red? Feed the failure tail back and call pi again. The evidence, not the model's memory, drives round i+1.

Read the scoreboard language: “P3: PASS green in 2” — rounds-to-green is a capability signal the harness records for free. oss120's whole lane averages ~2.

run_model_30.sh · the L3 template (verbatim)

```
for i in $(seq 1 8); do
  (cd "$ROOT/p3" && "$PYTEST" -q test_thing.py)
    >/dev/null 2>&1 &&
    { record "P3: PASS green in $i"; break; }

  F=$(cd "$ROOT/p3" && "$PYTEST" -q \
      test_thing.py 2>&1 | tail -n 25)

  run_pi "$ROOT/p3" "…Smallest fix.
    DO NOT edit test_thing.py.
    Failures:\n$F"
  [ "$i" = 8 ] && record "P3: FAIL after 8"
done
```

FAMILY B · FIX FAILING TESTS

P13 — mini-JSON parser — the fix-family wall

70%

fleet pass rate

L3 · iterate-until-green

THE TASK (verbatim prompt)

“Fix miniparse.py: parse(s) parses a JSON subset (null/true/false/int/float/string/array/object, nesting, whitespace) WITHOUT using the json module or eval. DO NOT edit test_thing.py.”

THE LOOP

6 rounds of pytest-tail feedback. The seed parser handles flat values; nesting and whitespace are where it's broken.

The separator: seven lanes failed — a real recursive-descent parser is the cleanest line between mid-tier and top-tier in Family B.

the shape that passes

```
def parse_value(s, i):
    i = skip_ws(s, i)
    if   s[i] == '{': return parse_obj(s, i)
    elif s[i] == '[': return parse_arr(s, i)
    elif s[i] == '"': return parse_str(s, i)
    elif s.startswith("null", i): …
    else:  return parse_num(s, i)

# every parse_* returns (value, next_i) —
# models that track index by mutation or
# regex-tokenize tend to die on nesting

# banned: import json · eval · ast helpers
```

Four more red-to-green drills

P11 · balanced parens — 91%

Stack discipline + the “ignore other chars” clause models forget. Push openers, match closers, empty stack at end.

```
is_balanced("a(b[c]{d})") → True
```

P20 · TTL-LRU cache — 91%

LRU + expiry where time comes ONLY from an injected clock(). Expired entries are misses. First taste of dependency-injected time.

```
TTLCache(cap, ttl, clock)
```

P22 · toposort + cycles — 96%

Kahn's algorithm or DFS; must raise on a cycle rather than loop forever. Friendliest of the six — one lane missed.

```
toposort({a:[b], b:[a]}) → ValueError
```

P29 · calculator, no eval — 87%

Precedence and parentheses via shunting-yard or recursive descent. eval() is banned and grepped for.

```
calc("2+3*(4-1)") → 11
```



Pattern: every Family B failure mode is either ignoring the evidence or over-fixing. “Smallest fix” is in the prompt because restraint is rarer than skill.

FAMILY C · NUMERICAL & ALGORITHMIC · 9 PROBLEMS

Family C — the bug is mathematics

Overflow, catastrophic cancellation, wrong recurrences, naive algorithms. Numerical literacy as a capability probe.

P4 61% softmax stability	**P8** 83% RK4 ODE integrator	**P12** 100% base-2…36 convert	**P14** 87% Dijkstra shortest path	**P18** 91% Welford variance
P21 91% interval merge	**P23** 100% prime-count sieve	**P26** 87% Newton square root	**P28** 96% determinant vs numpy	

Two poles: P12 and P23 are universal passes — mechanical correctness is solved. P4 (subtract-the-max softmax) fails 4 in 10 lanes including strong ones: knowing the stability trick is knowledge, not coding. The 100%-vs-61% gap inside one family is the benchmark working as designed.

FAMILY C · NUMERICAL & ALGORITHMIC

P4 — stable softmax — the fleet's IQ test

61%

fleet pass rate

L4 · oracle check

THE TASK (verbatim prompt)

> *“fast.py softmax overflows / disagrees with reference in check.py. Fix for numerical stability (subtract max before exp). DO NOT edit check.py. Diagnostic: <last 5 lines>”*

THE LOOP

Up to 6 rounds against an oracle: check.py compares fast.softmax to a trusted reference on brutal inputs (rows around ±1000) and prints maxerr.

Epilogue (2026-07-24): the oracle ITSELF had a defect — inkling's lone “FAIL” re-scored to PASS at maxerr 1.1e-16 after the fix. Even graders need graders. Full story in Part 5.

the one-line difference between 61% and 100%

```
# seed — overflows at x ≈ 1000:
def softmax(x):
    e = np.exp(x)          # inf / nan
    return e / e.sum(axis=-1, keepdims=True)

# fix — same math, shifted by the max:
def softmax(x):
    z = x - x.max(axis=-1, keepdims=True)
    e = np.exp(z)          # bounded ≤ 1
    return e / e.sum(axis=-1, keepdims=True)

# exp(x-c)/Σexp(x-c) == exp(x)/Σexp(x) —
# invariant math, finite floats
```

FAMILY C · NUMERICAL & ALGORITHMIC · L8 REFLECTION

P8 · P18 · P26 — fix it, then write down the lesson

Each prompt ends: “Append a one-line ROOT-CAUSE LESSON to ~/.pi/agent/memory.md.” The harness measures whether the agent actually did.

P8 · RK4 ODE integrator — 83%

Use RK4 and compute t = t0 + step·h — never accumulate t += h (drift). Convergence checked at two step sizes.

P18 · Welford variance — 91%

Online SAMPLE variance without catastrophic cancellation; constant streams must give exactly 0.

P26 · Newton square root — 87%

Full double precision, no math.sqrt; iterate to convergence, handle 0 and 1 without infinite-looping.

MEASURING REFLECTION

```
# harness, before & after the problem:
MB=$(wc -c < ~/.pi/agent/memory.md)  # bytes before
… loop runs …
record "P8: memory.md ${MB}->${MA} bytes"
P8: memory.md 2362->2471 bytes   # oss120: +109 = lesson written
```

Why it matters: agent memory only compounds if the agent actually writes to it. Byte-delta is a crude but honest probe of the reflection habit.

The mechanical four

P12 · base-N conversion — 100%

to_base(n, base) and parse_int(s, base) for 2–36, round-tripping against int(s, base). Zero and negatives included.

```
to_base(-255, 16) → "-ff"
```

P14 · weighted Dijkstra — 87%

heapq-based shortest path checked against a BFS reference oracle; unreachable handling per the checker.

```
shortest_path(g, src, dst) → cost
```

P21 · interval merge — 91%

Sort, then sweep; touching intervals count as overlapping. The classic interview question, graded by oracle.

```
[1,3]+[3,5] → [1,5]
```

P28 · determinant — 96%

N×N determinant matching numpy.linalg.det within tolerance — LU with pivoting or careful cofactor expansion.

```
det(A) ≈ np.linalg.det(A)
```

Family D — make it faster, prove it, stop when told

Correct-but-slow seeds. The loop measures real throughput each round and stops on target, plateau, or budget.

ANATOMY OF A MEASUREMENT LOOP

```
# the measurement loop (P5 shown; P15/P23 same)
best=0; stale=0; TARGET=5.0        # GFLOP/s
for i in $(seq 1 8); do
  check.py || fix-correctness-first  # gate!
  g=$(python3 bench.py --report)
  record "P5: round $i -> ${g} GFLOP/s"
  g >= TARGET  → PASS; break
  g > best     → best=g; stale=0
  else         → stale++
  stale >= 3   → STOP plateau; break
  run_pi "…does ${g}. Make ONE change to go
    faster. Stay correct vs check.py."
done                  # round 8 → STOP budget
```

Three exits, three meanings

- PASS — hit the throughput target while still passing check.py
- STOP plateau — 2–3 rounds without improvement; records best honestly
- STOP budget — ran out of rounds; best recorded, not a PASS
- **Correctness re-gates EVERY round: a fast wrong kernel never scores.**

FAMILY D · OPTIMIZE TO A TARGET

P5 — matmul — 0.2 → 33 GFLOP/s in one move

100%

fleet pass rate

L5 · measurement

THE TASK (verbatim prompt)

Round 1 (if incorrect): “kernel.matmul INCORRECT. Fix correctness (match numpy).” Then: “kernel.matmul does <g> GFLOP/s on 256×256 GEMM. Make ONE change to go faster (numpy vectorize / np.dot / @). Stay correct vs check.py.”

THE LOOP

Target 5.0 GFLOP/s on a 256×256 GEMM. Seed: triple Python loop (~0.2 GFLOP/s). One round of vectorization clears the bar by 6×.

“ONE change” is the pedagogy: it forces measure → hypothesize → change → re-measure, instead of rewrite roulette. Every lane that reached P5 passed it.

oss120's actual RESULTS.txt lines

```
[P5] matmul GFLOP/s
P5: round 2 -> 33.2675 GFLOP/s
P5: PASS 33.2675 GFLOP/s
```

```
# the seed (correct, glacial):
for i… for j… for k…:
    out[i][j] += a[i][k] * b[k][j]

# the one change that wins:
def matmul(a, b):
    return np.asarray(a) @ np.asarray(b)

# 0.2 → 33.27 GFLOP/s = 166× — BLAS says hi
```

The 253× one-liner, and its sieve cousin

P15 · oss120 transcript + winning shape

```
[P15] word-frequency top-k (Mtok/s)
P15: round 1 -> 0.1171 Mtok/s
P15: round 2 -> 29.6412 Mtok/s
P15: PASS 29.6412 Mtok/s   # 253×

# the change:
from collections import Counter
import heapq
def top_k(text, k):
    c = Counter(text.lower().split())
    return heapq.nlargest(k, c.items(),
        key=lambda kv: (kv[1], kv[0])) *
```

P23 · prime-count sieve (100%)

- Target 8.0 Mn/s counting primes below n — the seed trial-divides
- Winning move: Sieve of Eratosthenes on a bytearray, evens skipped, slice assignment for composites
- Same loop mechanics as P5/P15 — correctness gate, then throughput rounds
- In degraded-provider runs this family exposes throughput collapse: same code, ~800× slower measured — a provider fingerprint (Part 5)

*** the tie-break subtlety:** top-k orders by count then word — models that sort by count alone fail check.py before speed ever counts. Correctness first, always.

FAMILY E · TOURNAMENTS & CRITICS · 8 PROBLEMS

Family E — parallel selves and hostile reviewers

Best-of-N arenas, critic gates, and the two capstones. The most agentic — and the most separating — family.

P6 83% deterministic reduce + critic	**P9** 91% levenshtein tournament	**P10** 83% prime-sieve capstone	**P16** 87% thread-safe counter + critic
P19 91% fast-doubling fib tournament	**P24** 52% token-bucket limiter + critic	**P27** 100% overlap-count tournament	**P30** 91% sort-kernel capstone

The family thesis: sampling N independent attempts and selecting by measurement beats hoping one attempt is right — and a hostile self-review pass catches what generation misses. P24 (52%) is the suite's hardest problem; P27 (100%) its friendliest tournament.

P9 — levenshtein — best-of-4 arena

91%

fleet pass rate

L9 · tournament

THE TASK (verbatim prompt)

Four parallel pi calls, each in its own arena/cand_k/ dir: "Write levenshtein.py exposing levenshtein(a,b)→int edit distance. CORRECT for all inputs incl empty, as FAST as possible (two-row DP, early exit). Candidate #k distinct angle."

THE LOOP

score.py gates each candidate on 10 known pairs + 300 randomized checks vs a trusted DP reference — wrong on ANY → score 0. Survivors are timed; champion = max kops/s.

Selection is the safety net: cand_4 scored 0 (wrong code) and simply lost. A tournament converts per-sample variance into fleet-level reliability.

oss120's actual P9 arena

```
[P9] levenshtein tournament best-of-4
P9: cand_1 score=14.514
P9: cand_2 score=13.826
P9: cand_3 score=14.980
P9: cand_4 score=0      # incorrect → out
P9: PASS champion=cand_3 @ 14.980

# score.py, the gate before the race:
for a, b, d in KNOWN:
    if lev(a, b) != d: print("0"); return
# + 300 random pairs vs reference DP
# only THEN: timed kops/s
```

The hostile-reviewer pattern (P6 · P16 · P24)

run_model_30.sh · generator + critic (P16 shown)

```
# call 1 — the builder:
"Fix counter.py so class Counter with
 increment() is thread-safe … Address
 CRITIQUE.md if present. DO NOT edit verify.py."

# call 2 — the assassin (same model!):
"HOSTILE reviewer fresh eyes: inspect
 counter.py ONLY for data races / missing
 lock. Write findings to CRITIQUE.md.
 If zero real issues write exactly
 NO_ISSUES on first line."

# gate: NO_ISSUES on line 1  AND  verify.py green
```

Why two hats beat one

- Fresh context = fresh eyes: the critic call carries none of the builder's rationalizations
- CRITIQUE.md persists — the next builder round must address it
- The exit needs BOTH the critic's NO_ISSUES and a green verifier — self-satisfaction alone never passes
- Fallback after 5 rounds: verifier-only PASS, recorded as “critic not satisfied”

Targets: P6 — bit-identical parallel_sum regardless of chunking (math.fsum / fixed ordering) · P16 — a lock around shared state · P24 — the token bucket (next slide, bring a helmet).

FAMILY E · TOURNAMENTS & CRITICS

P24 — token bucket — the suite's hardest

52%

fleet pass rate

L6 · generator + critic

THE TASK (verbatim prompt)

> *"Fix limiter.py so TokenBucket(rate, capacity, clock) allows at most 'capacity' burst then refills at 'rate' tokens/sec using the injected clock(); allow(n=1)→bool." Plus a hostile critic on burst/refill/clock misuse.*

THE LOOP

Builder + critic rounds (×5). verify.py drives a fake clock: burst 10 at t=0 → exactly 5 allowed; +1.0 s → 5 more; +0.4 s → ~2 more.

We failed it too: our first hand-written fix died on PARTIAL: 1.4−1.0 = 0.399…96 in floats, so the 2nd token came up 4e-16 short. One 1e-9 epsilon later: OK P24. Eleven of 23 lanes never got there.

sandbox reproduction, 2026-07-24

```
# verify.py's third act — partial refill:
clk.advance(0.4)   # 5/s × 0.4s ≈ 2 tokens
a3 = sum(1 for _ in range(10) if rl.allow())
if a3 not in (2, 3): FAIL

# our first attempt — textbook, and wrong:
self.tokens = min(cap, tokens + dt*rate/per)
if self.tokens >= 1.0:     # 1.999…96 < 2.0!
PARTIAL: allowed 1 after 0.4s, expected ~2

# the fix — respect float reality:
if self.tokens >= 1.0 - 1e-9:
OK P24                          # exit 0 ✓

# three traps: burst cap · fractional refill
#              · injected clock (never time.time())
```

FAMILY E · TOURNAMENTS & CRITICS · L9/L10

Tournaments' greatest hits + the two capstones

P19 · fast-doubling fib — 91%

Exact bigint fib(200000), O(log n) fast-doubling required — naive linear DP times out in score.py. Best-of-4.

```
P19: PASS champion=cand_3 @ 202.34
```

P27 · overlap count — 100%

count_overlapping('aaaa','aa') = 3 — str.count doesn't overlap, so slide a window or loop str.find. Everyone passed.

```
P27: PASS champion=cand_3 @ 0.2780
```

P10 · prime-sieve capstone — 83%

Best-of-3 sieve to 2M + a critic that may FIX and must write VERDICT.txt (CLEAN or FIXED) before benching.

```
P10: cand_3 verdict=CLEAN score=38.21
```

P30 · sort-kernel finale — 91%

Best-of-3 my_sort + critic + bench. Infamous for a grader-side hang that faked failures until SCORE_TIMEOUT existed (Part 4).

```
P30: PASS champion=cand_3 @ 6.7359
```



That's all 30. Ten minutes of coffee, then we open the harness that kept these honest across 26 fleets.

1:45 – 1:55

Break — 10 minutes

When we return: the unglamorous 80% — timeout guards, path clamps, resume logic, and how 13 lanes ran in parallel without eating each other.

Break trivia: the fastest full-fleet lane finished all 30 problems in 9 minutes — faster than this break. The slowest took 7 h 36 m for 19/30. Speed is a capability axis.

PART 4 · 1:55 – 2:15 · ~20 MIN

Harness engineering

528 lines of bash stand between “we ran some models” and “these numbers are true.”
Every guard in this part was paid for with a real incident.

env hygiene | two timeout guards | path clamp | resume | fleet mode | tool-call Babel

THE RUNNER

run_model_30.sh at 10,000 feet

One sequential lane: stage fresh, guard everything, loop per problem, verdict from exit codes, summarize separately.

1 · hygiene	unset PYTHONPATH, pin node/python paths, locate gtimeout (FATAL without it)
2 · guards	PT = gtimeout -k 15 $PI_TIMEOUT pi · SC = gtimeout -k 5 $SCORE_TIMEOUT
3 · stage	rm -rf runs30/<tag> (unless resuming), copy fresh verifiers + seeds for p1…p30
4 · resume gate	PI30_RESUME=1 → skip problems already PASS; re-run FAILs
5 · 30 problem blocks	each: prompts → run_pi (clamped, flattened) → grader exit code → record
6 · summarize	SUMMARY.txt written to a SEPARATE file (never append P-lines back into RESULTS)

```
Invocation:  PI_TIMEOUT=360 bash run_model_30.sh <provider> <model-id> <tag>
```

ENVIRONMENT HYGIENE

The PYTHONPATH leak that failed every model

run_model_30.sh · lines 7–18 (condensed)

```
# the shell exported:
PYTHONPATH=~/Library/Python/3.9/site-packages

# which contains a bare py.py that SHADOWS
# the real `py` package. pytest 6.2.4 does:
from py.test import console_main
ModuleNotFoundError: No module named 'py.test';
'py' is not a package

# → EVERY pytest-based problem (12 call sites)
#   auto-FAILED for ALL models → false 0/N lanes

# the fix, top of run_model_30.sh:
unset PYTHONPATH
export CONDA_NO_PLUGINS=true
```

Why this is the scariest bug class

- It fails every model equally — so it looks like “all models are bad at pytest problems”, not like a harness bug
- It rode in via login shells the pi bash tool spawns — the same leak also crashed conda's urllib3 on every prompt
- Detected only because a 0/N triggered the Zero-Score Rule audit
- **Fix is two lines. Finding it took a day.**

GUARD #1

PI_TIMEOUT — no call may wedge the lane

run_model_30.sh · the PT wrapper

```
# pi has NO internal request timeout.
# 2026-07-03: a hung endpoint request wedged a
# sequential lane 11 hours on P26.

GT=$(command -v gtimeout || command -v timeout)
PI_TIMEOUT="${PI_TIMEOUT:-300}"   # s per pi call
PT="$GT -k 15 $PI_TIMEOUT $PI"
[ -z "$GT" ] && { echo "FATAL: no gtimeout…"
  ; exit 2; }

# -k 15 : if pi ignores SIGTERM,
#         SIGKILL follows 15 s later
# exit 124 = timed out → problem records FAIL,
#          lane marches on
```

Design points

- Every pi invocation goes through $PT — there is no unguarded path
- Tunable per run: PI_TIMEOUT=600 for slow providers, 360 for fleets
- A timeout is a clean FAIL, not a crash: the loop's other rounds still count
- Same medicine later applied to graders (next slide) — hangs come from both sides

GUARD #2

SCORE_TIMEOUT — the 6.5-hour poison candidate

run_model_30.sh · the SC wrapper

```
# 2026-07-06: ornith-9b's P30 candidate was a
# radix sort with an infinite loop. score.py
# imports candidates & runs them IN-PROCESS…
# → the bench hung a lane for ~6.5 hours

SCORE_TIMEOUT="${SCORE_TIMEOUT:-120}"
SC="$GT -k 5 $SCORE_TIMEOUT"

# wraps ALL 5 benchmark-scored problems:
#   P9 · P10 · P19 · P27 · P30
f=$(cd "$ROOT/p30" && $SC "$PY" score.py \
    "$d/sort.py"); [ -z "$f" ] && f=0

# SIGKILL at timeout → empty stdout → f=0 →
# the poison candidate simply LOSES the arena
```

The payoff

- Both stuck lanes re-ran P30 after the guard landed → ornith-9b 30/30, uic-qwen36-35b-a3b 30/30
- Verified against the actual hung kernel: SIGKILL fires, candidate scores 0, lane proceeds

Principle: untrusted code (model output!) never runs unguarded — even inside your own grader. The grader is part of the attack surface.

ROBUSTNESS LAYER 1

PATH_CLAMP — keep artifacts where the grader looks

run_model_30.sh · PATH_CLAMP (verbatim, trimmed)

```
# weaker tool-callers INTERMITTENTLY emit paths
# embedding the cwd — pi joins them onto cwd:
write("private/tmp/…/p12/baseconv.py")
# → artifact nests a level deep → verifier
#   checks $ROOT/p12/baseconv.py → false FAIL

# fix: appended to EVERY call's system prompt —
PATH_CLAMP="CRITICAL FILE-PATH RULE: when using
  the write or edit tools, ALWAYS pass a bare
  relative filename such as 'cli.py'. NEVER pass
  an absolute path. NEVER include /tmp, /Users,
  /private, or any part of the working directory.
  The tool already runs in the correct directory.
  … your work is lost."
```

Why prompt, not patch?

- Tools stay ON (Rule 3) — no retreat to --no-tools
- Injected via --append-system-prompt on every run_pi call
- Across 30 problems, ONE bad path can tank a lane to ~0 — the clamp turns a lane-killer into a non-event
- Layer 2 (next slide) catches the ones that slip through anyway

ROBUSTNESS LAYER 2

flatten_nested() — and the bug inside the fix

run_model_30.sh · flatten_nested()

```
# after every pi call: lift nested *.py back up
find "$dir" -mindepth 2 -name '*.py' | while …
  case "$f" in
    "$dir"/work/*) continue ;;  # ← THE FIX
  esac
  [ ! -e "$dir/$base" ] && mv "$f" "$dir/$base"

# FIX (2026-07-05): NEVER flatten work/ subtrees.
# Fan-out problems (P7/P17/P25) LEGITIMATELY put
# modules at work/<name>/backend.py — the original
# flatten lifted those up, DESTROYING the module
# the verifier imports → FALSE FAIL for EVERY
# model on all three fan-out problems.

# also: never clobbers an existing top-level file
```

Lessons stacked here

- Robustness code needs its own tests — the helper built to prevent false FAILs caused them
- “Every model fails problem X” → suspect the harness, not 13 models (the Zero-Score reflex again)
- The two layers compose: clamp prevents most nesting, flatten repairs the rest, work/ stays sacred

THE ATOM

run_pi() — one line, every safeguard

run_model_30.sh · the whole contract in two lines

```
run_pi() { ( cd "$1" && $PT -p $PROV $NS --append-system-prompt "$PATH_CLAMP" "$2" )
           >/dev/null 2>&1; flatten_nested "$1"; }
```

`( cd "$1" && … )`	subshell: pi runs INSIDE the problem dir; cwd never leaks back
`$PT`	gtimeout -k 15 $PI_TIMEOUT pi — guard #1 baked into the binary path
`$PROV · $NS`	--provider X --model Y · --no-session -na — lane identity, zero state, no local-config trust
`--append-system-prompt`	the PATH_CLAMP rides on every single call
`>/dev/null 2>&1`	model stdout is NOISE — verdicts never read it (Rule 1, enforced by plumbing)
`flatten_nested "$1"`	repair nesting immediately, before any grader looks

DURABILITY

Resume: crash-safe lanes with skip_gate

run_model_30.sh · PI30_RESUME_PATCH

```
# PI30_RESUME=1 → keep RESULTS.txt, skip PASSes
declare -A PI30_DONE=()
while read line; do
  pn=$(sed -nE 's/^(P[0-9]+): PASS.*/\1/p')
  [ -n "$pn" ] && PI30_DONE[$pn]="$line"
done < "$PRIOR_RESULTS"

skip_gate() {
  if resuming && already PASS; then
    echo "${PI30_DONE[$1]}  [resumed-skip]"
    return 0   # skip this problem
  fi; return 1      # run it
}

if ! skip_gate "P7"; then … fi
```

Semantics that matter

- PASS is durable — a crashed lane resumes without re-earning green
- FAIL is NOT durable — failed problems re-run on resume (they deserve another shot at the full budget)
- Cached lines re-emit tagged [resumed-skip] — provenance visible in RESULTS.txt
- Default (unset) = fresh full run: rm -rf and start clean. Resume is opt-in.

THE LEDGER

RESULTS.txt: append-only truth, summarized carefully

runs/oss120-cels-vllm014/
RESULTS.txt (excerpt)

```
==== oss120-cels-vllm014 START Jul 4 08:52 ====
[P1] stats CLI
P1: PASS
[P3] Roman numerals iterate-until-green
P3: PASS green in 2
[P5] matmul GFLOP/s
P5: round 2 -> 33.2675 GFLOP/s
P5: PASS 33.2675 GFLOP/s
[P9] levenshtein tournament best-of-4
P9: cand_3 score=14.980
P9: PASS champion=cand_3 @ 14.980
==== oss120 DONE 09:01 ====

SCORE: 30/30 passed   ← 9m 10s wall-clock
```

Counting rules (learned the hard way)

- SUMMARY.txt is a SEPARATE file — an early version appended the summary into RESULTS.txt, and the next grep double-counted every P-line
- Resume mode appends — so raw grep -c PASS over-counts; true scores dedup by LAST verdict per problem
- Multi-round lines (round N → …) are progress, not verdicts; only “PN: PASS/FAIL/STOP” lines score
- A 2×-coverage run once read as “18/10” — attempts vs problems. Canonicalize per-problem, always.

SCALE OUT

Fleet mode: 13 lanes in parallel, politely

harness/fleet_launch.sh (condensed)

```
MODELS=(gemma4-e2b gemma4-e4b gemma4-12b
  gemma4-31b ornith-9b devstral-small-2
  qwen36-27b uic-ornith-9b … )  # 13 total

for m in "${MODELS[@]}"; do
  PI30_RESUME=1 PI_TIMEOUT=360 \
  setsid bash run_model_30.sh \
    "$PROVIDER" "$m" "fleet-$m-$STAMP" \
    > "$log" 2>&1 &
  setsid bash -c "… heartbeat …" &
done; wait

# setsid: each lane gets its OWN process group —
# SIGTERM to the launcher cascade-kills NOTHING
```

Fleet furniture

- One aggregator provider (chiatta00-agg) fronts all 13 models — the launcher just varies --model
- Heartbeat sidecar per lane: greps the latest [Pn] marker + pass/fail counts into <model>.cur every 5 s
- PI30_RESUME=1 by default — a re-launch continues every partial lane instead of burning finished work

OBSERVABILITY

The live fleet board (and its 5-minute cron)

harness/fleet_report.sh
output (illustrative)

```
$ bash fleet_report.sh
pi-30 FLEET status  21:42:07 CDT
---------------------------------------
gemma4-12b      [P17] serializer fan-out
                          pass=14 fail=2
ornith-9b       [P29] calculator   pass=26 fail=2
qwen36-27b      DONE          pass=30 fail=0
uic-laguna-xs2  [P24] token bucket pass=21 fail=1
…
---------------------------------------
4/13 lanes complete

# same board piped to Telegram every 5 min —
# glanceable fleet state from a phone
```

Ground-truth discipline

- The board reads only RESULTS.txt + process liveness — never the models' own claims
- A background-process tracker once spammed false “exited (code None)” alerts while PIDs were alive. Standing rule: ps -p <pid> + RESULTS.txt are truth; notifications are hints
- Cheap observability (grep + printf) outlived every fancier idea

THE DEEP PROBLEM

Tool-calling Babel: dialects vs parsers

pi sends OpenAI tool schemas. Models answer in their NATIVE dialect. The serving runtime must translate — or the loop dies silently.

THE DIALECT PROBLEM

```
# what qwen3-coder actually emits:
<function=write><parameter=path>
cli.py</parameter>…
# without a parser: raw text → pi sees
# NO tool_call → no file → FAIL
```

vLLM: the per-model parser flags

```
--enable-auto-tool-choice --tool-call-parser P
P = qwen3_xml · hermes · llama3_json · mistral · openai (gpt-oss)
· glm47  + matching --chat-template
```

Ollama	per-model lottery: some models return structured tool_calls, others emit raw XML it won't parse	**UNRELIABLE for pi tool-use**
LM Studio	returns correct structured tool_calls (JIT-loads models on demand)	**WORKS**
vLLM	the proper fix — explicit parser + template per model family	**WORKS (config required)**
llama.cpp	works IF you pass --jinja AND an explicit chat template (Part 5 tells the scar story)	**WORKS (mind the template)**

ESCAPE HATCHES

The universal fallback, and the aggregator trick

Code-extraction harness (works on ANY model)

- pi --no-tools + “reply with ONE fenced code block”
- extract_code.py pulls the block and writes the artifact itself; verdict still from the verifier
- Confirmed: models with broken tool parsing score P1–P3 PASS via extraction — capability was there, plumbing wasn't
- Kept as a labeled diagnostic set — never merged with tools-ON scores (Rule 3)

LiteLLM aggregators + SSH tunnels

```
model_list:
- model_name: "spark/gpt-oss-120b"
  litellm_params:
    model: "openai/gpt-oss:120b"
    api_base: "http://<node>:11434/v1"
```

- One OpenAI-compatible front door for a whole cluster; pi registers the aggregator once, --model routes
- Caveat from the fleet: a proxy path can reproducibly drop specific problems a native path passes — measure the proxy too (it's a lane!)

LAB 2 · 2:15 – 2:30 · 15 MIN

Run your own lane

Clone pi-30, port four paths to your laptop, run single problems by hand — then let the harness run a mini-lane against your model.

clone & port | one problem by hand | harness mini-lane | read your RESULTS.txt

Timebox 15 minutes. A single problem takes ~1–3 min against a decent endpoint; a full 30-problem lane runs 10 min–3 h depending on provider — start one and let it cook through Part 5.

LAB 2 · STEP 1

Clone, then port four paths

PORTING CHECKLIST

```
$ git clone \
  https://github.com/rick-stevens-ai/pi-30
$ cd pi-30/harness

# top of run_model_30.sh — point these at
# YOUR machine (they ship as the author's):
PI="$(command -v pi)"
PY="$(command -v python3)"
PYTEST="$(command -v pytest)"
SRC="$PWD/.."; ROOT="$SRC/runs30/$TAG"

# macOS: the harness needs gtimeout —
$ brew install coreutils
# (Linux: /usr/bin/timeout is found instead)
```

Why paths are pinned at all

- Verifiers must run under a Python that HAS numpy + pytest — not whatever the model's bash finds first
- Pinning pi's absolute path keeps the gtimeout wrapper honest ($PT bakes them together)
- Sanity check before the lab: python3 -c "import numpy" && pytest --version

One problem by hand, then a real lane

THE WHOLE GAME IN 5 COMMANDS

```
# — step 2: P1 manually, so nothing is magic —
$ mkdir -p /tmp/lab-p1 && cd /tmp/lab-p1
$ cp <repo>/problems/p1/verify.py .
$ pi -p --no-session -na \
    --provider openrouter --model <id> \
    "Write cli.py: reads whitespace-separated
     numbers from STDIN, prints count= min=
     max= mean= median= stdev= (SAMPLE stdev).
     Stdlib only. Exit 0."
$ python3 verify.py && echo PASS || echo FAIL

# — step 3: the harness runs a full lane —
$ PI_TIMEOUT=360 bash run_model_30.sh \
    <provider> <model-id> my-first-lane
$ tail -f ../runs30/my-first-lane/RESULTS.txt
```

While it runs, watch for

- “green in N” on the iterate problems — your model's feedback-use signature
- P5/P15 round lines — throughput jumps of 100×+ when vectorization lands
- P9 arena lines — four candidates, scores, a champion (or a 0)
- Any problem stuck > PI_TIMEOUT — the guard will FAIL it and move on; that's the system working

Read your lane like an operator

FIELD GUIDE TO A RESULTS.txt

```
[P4] stable softmax vs oracle
P4: PASS in 3        # rounds used: 3 of 6
[P5] matmul GFLOP/s
P5: round 1 -> 0.9143 GFLOP/s
P5: STOP plateau best=4.2110
   # improved, stalled 3 rounds — honest best
[P9] levenshtein tournament best-of-4
P9: cand_2 score=0      # wrong → eliminated
P9: PASS champion=cand_1 @ 11.203
[P17] serializer fan-out
P17: FAIL
   # → ls runs30/<tag>/p17/work/ — which
   #   backend is missing? assembly or parts?

grep -cE "^P[0-9]+: PASS" RESULTS.txt  # score
```

Verdict grammar

`PASS`	grader exit 0 — the only success
`PASS green in N`	+ rounds-to-green telemetry
`PASS champion=…`	tournament winner + its score
`STOP plateau/budget`	measurement loop's honest best
`FAIL / FAIL after N`	budget exhausted, still red
`[resumed-skip]`	carried from a prior run

Reconvene 2:30 — lanes keep cooking into Part 5.



PART 5 · 2:30 – 2:45 · ~15 MIN

The zero-score files

Five debugging whodunits from the fleet. Four zeros that were never the model — and the one lane that really was. Every story ends in a durable lesson.

streaming bug | missing template | phantom model | the real ceiling | the guilty oracle

05

WAR STORY 1 · THE ZERO-SCORE FILES

oss120: the invisible torrent

0/30 → 30/30

lane score, before → after the fix

Symptom. gpt-oss-120b — a frontier open model — scores 0/30. Every tool-call-heavy problem hangs to timeout.

Investigation. server-side, the engine is GENERATING at 130–190 tok/s for 90+ seconds (Running:1)… but the SSE stream surfaces nothing past chunk 0. Non-streaming curl works 100%.

Root cause. vLLM 0.13.0 streaming-flush bug in the openai_gptoss reasoning parser — pi's streaming tool-call client waits forever on deltas that never flush.

Fix. vLLM 0.14.1 in a cloned env; cut the endpoint over; prod env left pristine as rollback. Reproducer went 1/5 hanging → 5/5 clean (chunks=47, finish=tool_calls).

the tell

```
# engine view (healthy!):
Running: 1 · 130–190 tok/s · 90s+

# client view (starving):
SSE chunk 0 … [nothing ever again]

# non-stream curl: fine — the bug
# lives ONLY in streaming flush

# after vLLM 0.14.1:
chunks=47 finish=tool_calls dt=0.6s
full sweep: 30/30 in 9 minutes
```

Durable lesson when a streaming agent hangs, compare engine-side generation vs client-side deltas FIRST. The gap between them is a serving bug, not a model. Bonus rule: maxTokens (client budget) ≠ server context — know which knob you're turning.

WAR STORY 2 · THE ZERO-SCORE FILES

uic-ornith-9b: the template that wasn't there

0/30 → 28/30

lane score, before → after the fix

Symptom. A 9B reasoning model 0/30 via the aggregator — zero code artifacts, whole “run” fizzles in 4m36s. Trivial prompts (HELLO) work, masking it.

Investigation. On any real task: finish=stop, tool_calls=None, content='\n\n'. The model burned its ENTIRE output budget inside <think>…</think> (793–1069 tokens), then emitted nothing.

Root cause. the Ollama Modelfile used a generic ChatML template — no reasoning handling, no budget. And Ollama GGUF blobs carry NO chat template of their own, so tool-call parsing silently never happened.

Fix. re-serve via llama.cpp llama-server with an explicit template + reasoning controls (right). Fresh full run: a real 28/30 in 1h36m.

the fix (llama.cpp)

```
llama-server \
  --jinja \
  --chat-template-file \
    Ornith-1.0-9B/chat_template.jinja \
  --reasoning-format deepseek \
  --reasoning-budget 2048

# verified end-to-end after:
finish=tool_calls · real write_file
reasoning separated, no leak

# durable: --jinja ALONE silently
# fails without an explicit template
```

Durable lesson Ollama blobs carry no chat template — --jinja needs an explicit --chat-template-file or tool_calls silently fail. And reasoning models need a reasoning BUDGET, or thinking eats the entire response.

WAR STORY 3 · THE ZERO-SCORE FILES

nemotron-super: benchmarking a ghost

0/27 → 26/30

lane score, before → after the fix

Symptom. nemotron-3-super-120B scores 0/27 on a spark node. Direct probes of the endpoint… 404.

Root cause. the ~230 GB model was NEVER RESIDENT — the host has 119 GB of RAM. Every single call had failed. The lane benchmarked an empty socket.

Fix. re-route the same model to a real endpoint (OpenRouter free): 26/30 — best score in its family, ~7 points over its locally-serveable siblings.

Aftermath. harness rule: probe the endpoint with a trivial completion BEFORE staging 30 problems. A 404 should cost seconds, not a lane.

the arithmetic that solved it

```
model weights:   ~230 GB
host RAM:         119 GB
resident:         never

$ curl <node>:11434/v1/models
404 — model not found

# rerouted lane, same harness:
nemotron-super @ OpenRouter: 26/30
nano-30B  local:            19/30
33B Omni  local:            16/30
```

Durable lesson a lane's first artifact should prove the model EXISTS. 'Phantom endpoint' is embarrassingly common in home-lab fleets — and it wears a 0/N exactly like a dumb model does.

WAR STORY 4 · THE ZERO-SCORE FILES

glm-4.7-flash: the zero that was (mostly) real

0/23 → 1/28

after BOTH harness bugs were fixed

Symptom. 0/23 — but by now the rule says: audit the stack before blaming the model.

Layer 1 (harness). path-nesting: glm wrote cwd-prefixed paths, artifacts nested, verifiers missed them. Fixed with PATH_CLAMP + flatten_nested — the Part 4 machinery was BUILT during this audit.

Layer 2 (env). the PYTHONPATH py.py shadow crashing pytest — fixed at the root in the profile + belt-and-suspenders unset in the runner.

Verdict. clean re-run with both fixed: 1/28. Broken, incomplete code — a genuine capability ceiling. Retired; its successor glm-5.2 scores 29/30.

why this one story matters most

```
# the rule is NOT 'every zero is infra'
# the rule is 'PROVE it isn't, first'

audit found: 2 real harness bugs
fixed both  → re-ran clean
result: 1/28 — the model, truly

# and the audit's byproducts protected
# every OTHER lane in the fleet:
PATH_CLAMP · flatten_nested ·
PYTHONPATH hygiene
```

Durable lesson the Zero-Score Rule doesn't protect bad models — it protects the benchmark. Clearing the infrastructure is what makes a residual failure PUBLISHABLE as capability.

WAR STORY 5 · THE ZERO-SCORE FILES

P4's oracle: the grader was guilty

29/30 → 30/30

inkling, canonical → effective

Symptom. inkling (Thinking Machines, via OpenRouter) rips through Fleet-30 in 24 minutes… 29/30. The lone miss: P4, “FAIL after 6”.

Doubt. P4 already had a reputation (61%, the discriminator). But inkling's failed attempts LOOKED right. So: audit the oracle, not just the candidate.

Root cause. the P4 reference oracle itself had a defect. Re-scored against the corrected oracle: PASS, maxerr = 1.1e-16 — machine precision. The model had been right.

Bookkeeping. canonical 29/30 retained for provenance; effective 30/30 recorded; original output preserved in RESULTS.txt.bak-p4rescore.

the rescore

```
P4 (original oracle):  FAIL after 6
P4 (fixed oracle):     PASS
maxerr = 1.11e-16   # double-precision
                    #   machine epsilon!

# provenance kept:
RESULTS.txt.bak-p4rescore

# score bookkeeping:
canonical: 29/30   effective: 30/30
```

Durable lesson verifiers are code; code has bugs. When a strong lane fails EXACTLY one problem at machine precision, audit the oracle. (And keep the pre-rescore artifacts — score changes need provenance too.)

EPILOGUE · A SLOW-MOTION ZERO

Free-tier gravity: same weights, different product

oss120	local DGX · vLLM	30/30	9m
oss120	ALCF Sophia	29/30	—
oss120	OpenRouter free	14/30	63m
oss20b	spark · Ollama	27/30	57m
oss20b	ALCF Sophia	20/30	—
oss20b	OpenRouter free	6/30	63m

Degradation fingerprints

- Genuine multi-retry failures (“FAIL after 5/6”) — not crashes
- Measurement problems ~800× slower: P23 plateaus at 0.15 Mn/s vs ~127 on a healthy path
- 6–7× wall-time inflation on the same problem set
- **Recorded as PROVIDER-quality results — never quoted as the model's score**

Rule 6, earned: identical gpt-oss weights span 6/30 → 30/30 purely by serving path. “What model?” is half a question — always ask “served how?”

SYNTHESIS

The 0/N debugging playbook

Five stories compress into one flowchart. Laminate accordingly.

1	**Probe the endpoint raw**	trivial curl completion — does the model even exist? (Story 3)
2	**Check tool_calls end-to-end**	one real task: finish=tool_calls? or prose/empty? (Story 2)
3	**Compare stream vs non-stream**	engine generating but client starving = flush bug (Story 1)
4	**Audit template & reasoning budget**	explicit chat template? thinking bounded? (Story 2)
5	**Scrub the environment**	PYTHONPATH, shadowed packages, conda leaks (Story 4)
6	**Suspect your own graders**	run a known-good solution through the verifier (Story 5)
7	**Only THEN blame the model**	and publish the residual with the audit trail attached (Story 4)

PART 6 · 2:45 – 2:58 · ~13 MIN

What 26 lanes taught us

The integrated master table, four charts, and five findings that survive contact with the data. All regenerable from runs/ in the repo.

leaderboard | speed frontier | problem difficulty | provider gravity | 5 findings

THE FLEET

Five serving paths, one harness

local dgx

DGX endpoints incl. vLLM lanes — home of oss120's 9-minute 30/30

5 lanes

nVIDIA A100

8×A100-80GB DGX — the MoE playground (laguna, qwen36-a3b, ornith)

7 lanes

Intel PVC

Data Center GPU Max 1550, one 64 GB tile per dense model, via aggregator

7 lanes

spark · Ollama

DGX Spark nodes serving locally — capable but 4–8× slower wall-clock

5 lanes

OpenRouter

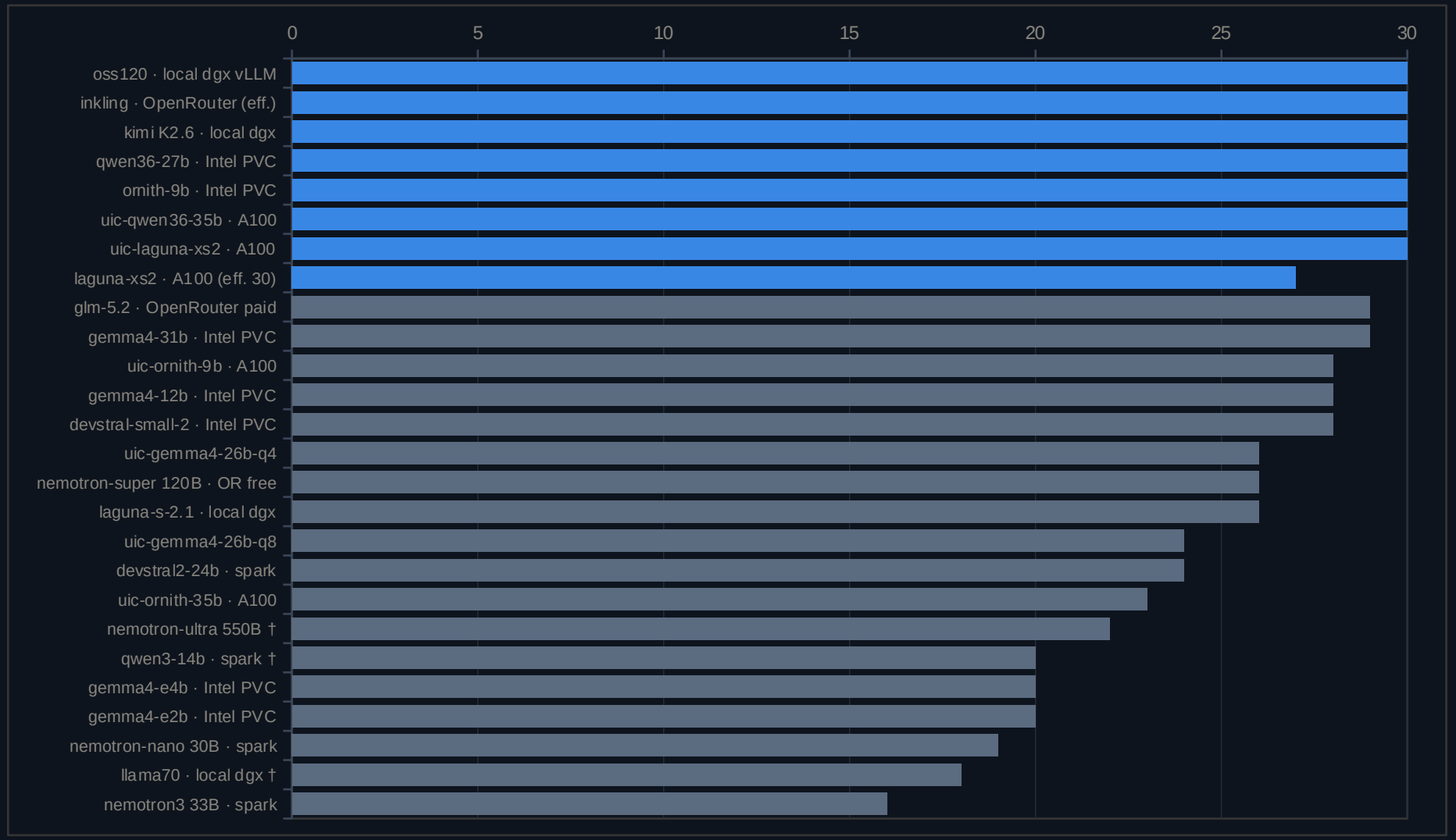
hosted API — free tier preferred, paid tagged; quality varies wildly

4 lanes

Read the table that follows as: 26 unique (model × backend) lanes, deduped, in score order — the same model on two backends stays two rows, because Part 5 proved the backend is half the product.

THE MASTER TABLE, DRAWN

All 26 lanes in score order



8

clean 30/30 lanes (blue bars)

9B

smallest model with a perfect score

16

floor: a 33B that wastes a vision stack

† partial coverage (hung or stopped early): scores shown as recorded / problems reached.

THE PODIUM

Eight clean 30/30 lanes — and their price tags

oss120 (gpt-oss-120b)	local dgx · vLLM 0.14.1	**9m**	*the benchmark's bullet train*
inkling (Thinking Machines)	OpenRouter	**24m**	*effective — P4 oracle rescore, maxerr 1.1e-16*
uic-laguna-xs2 (33B.A3B MoE)	nVIDIA A100 · aggregator	**49m**	*clean cold pass, single GPU*
uic-qwen36-35b-a3b (MoE)	nVIDIA A100 · aggregator	**52m**	*30/30 after the SCORE_TIMEOUT fix*
laguna-xs2 (direct)	nVIDIA A100 · ik_llama.cpp	**56m**	*27/30 single-shot → effective 30/30*
qwen36-27b	Intel PVC · 1 tile	**1h22m**	*dense 27B, clean*
ornith-9b (reasoning)	Intel PVC · 1 tile	**3h05m**	*NINE billion params, perfect score*
kimi (Kimi-K2.6)	local dgx	**3h18m**	*frontier-class, clean*

Spread inside perfection: 9 minutes to 3h18m for the same 30/30 — throughput is the tiebreaker that decides who gets production work.

THE USABILITY FRONTIER

Score vs wall-clock — where lanes earn real work



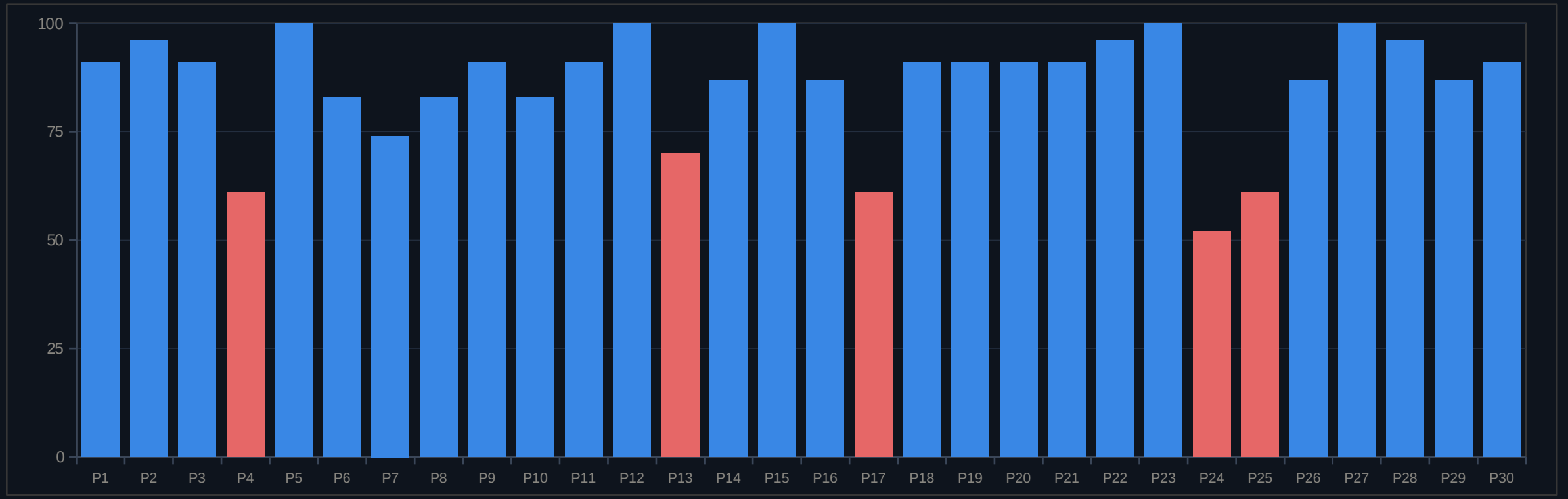
Operator's cut

- an 18/30 that takes 7 h is not a Kanban-worker candidate at any score
- speed separates equals: three 30/30s differ by 20×
- partial/hung lanes excluded (no honest x-value)

PROBLEM DIFFICULTY

Fleet pass rate, P1 → P30

red = the five separators



P24 token bucket · 52% | P4 softmax · 61% | P17 codec · 61% | P25 pipeline · 61% | P13 JSON parser · 70% | 100%: P5 · P12 · P15 · P23 · P27

Shape of the curve: mechanical competence is table stakes (five problems at 100%); the tail is concurrency, numerical insight, and multi-part composition.

THE SEPARATORS

What the five hardest problems have in common

P24 · 52%

token bucket + critic

three stacked traps: burst cap, fractional refill, injected clock — then a self-review gate on top

P4 · 61%

softmax stability

a one-line KNOWLEDGE test — you either know subtract-the-max or you don't

P17 · 61%

codec round-trip

a 2-function contract held across 3 import targets

P25 · 61%

pipeline compose

assembly of parallel parts — even glm-5.2 slipped here

P13 · 70%

JSON parser

sustained recursive-descent implementation, no library escape hatch

Injected time & concurrency

P24, P20, P16 cluster at the bottom — models that write Dijkstra flawlessly mishandle “refill r tokens/sec via clock()”

Insight beats syntax

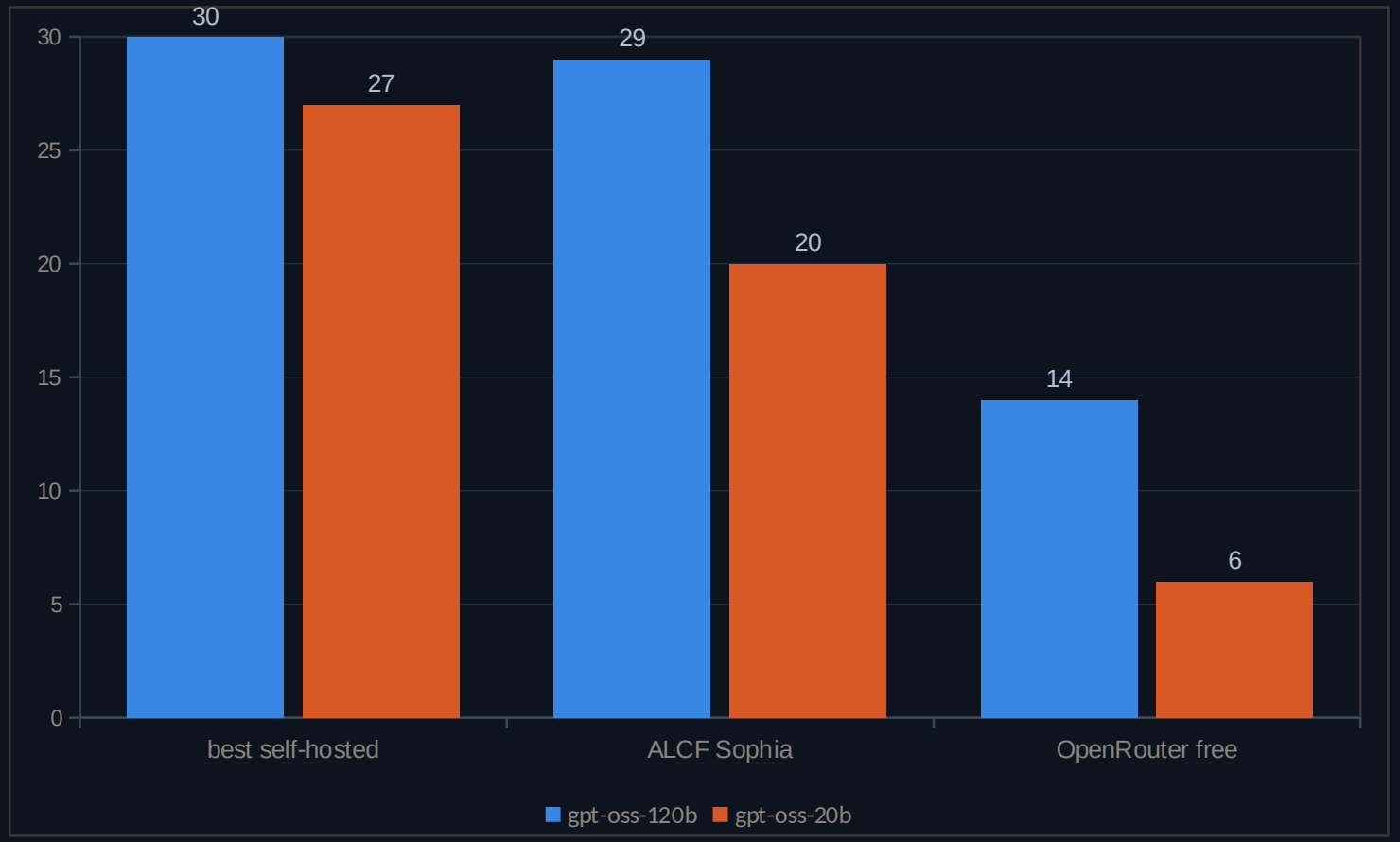
P4 and P13 fail on knowing-the-trick and sustained-structure — not on Python fluency

Composition is fragile

P7 → P17 → P25: each added contract multiplies failure odds; assembly is the skill scale doesn't buy

PROVIDER GRAVITY, QUANTIFIED

Identical weights, three serving paths



Same model. Same harness.

- 120b: 30 → 14. 20b: 27 → 6. The free tier is a different product wearing the same name
- ~800× slower measured throughput on P23; 6–7× wall-time inflation
- Not malice — throttling/truncation economics. But your eval must SEE it
- **Canonical scores: oss120 = 30/30 (self-hosted) · oss20b = 27/30 (spark)**

SIZE & ARCHITECTURE

Scale is not the gate — but it isn't nothing

9B = 30/30

ornith-9b runs the table on a single Intel PVC tile

Tool-use training + a correct serving stack beat parameter count. A 550B in this same fleet hung at P27; a 70B scored 18/29.

Size is not the gate.

33B > 120B

laguna-xs2 (33B-active MoE, one A100) outscores nemotron-super 120B

Effective 30/30 vs 26/30 — best perf-per-active-param in the fleet. Meanwhile or-llama33-70b bottoms out its set entirely.

Agentic competence ≠ raw scale.

26 > 19 > 16

nemotron-3 at 120B > nano-30B > 33B Omni — same family, one harness

WITHIN a family, scale tracks competence monotonically (~7 points frontier-to-local). The 33B 'Omni' pays a tax for an unused vision stack.

Scale helps — inside a family.

TRIANGULATION

Three independent sets, one headline

Fleet-30

P1–P30 · this tutorial

oss120 30/30 · kimi 30/30 · 8-lane perfect club

aggB-10

P1–P10 · independent operator & harness

gpt-oss-120b 10/10 · next best 9/10

mesh-extract-10

10-problem subset · spark mesh, extraction harness

gptoss20b 10/10 · qwen36-27b 10/10

- **gpt-oss family tops all three sets, under three different harnesses and owners — the one fully triangulated conclusion.**
- Sets are SEGMENTED: different numbering/subsets — never merged into one matrix (the provenance trap from slide 8).
- Cross-check bonus: a proxied route reproducibly dropped P2/P7 that the native route passed (−2) — even proxies are lanes.
- kimi bifurcates by version and set (K2.6: 30/30 Fleet-30 · k2.5: 4/10 aggB-10) — version pins matter as much as provider pins.

TAKE IT HOME · EXERCISE

Author P31 — the checklist

The best way to own this material: add a problem. Here's the quality bar the existing 30 meet.

1. **Seed fails, provably**
   run the verifier against your seed and demand a non-zero exit — before any model sees it

2. **Exit codes only**
   verifier asserts behavior; prints are decoration. No grading by prose, ever

3. **Pick a ladder rung**
   single-shot? iterate? tournament? — the loop shape is part of the problem design

4. **Feedback that teaches**
   pipe the diagnostic tail into retry prompts (bounded: tail -15, not the firehose)

5. **Guard everything**
   wrap the model call in $PT and any candidate execution in $SC — assume hostile infinite loops

6. **Forbid grader edits**
   say it in the prompt AND re-stage pristine graders each run

7. **Place it on a theme**
   the separators cluster on time/concurrency, insight tricks, composition — extend those veins

8. **Calibrate**
   run 3+ lanes of different strength; a problem everyone passes (or nobody) ranks nothing

THE DISTILLATION

Ten commandments of agent-loop benchmarking

1. Verdicts from exit codes, never prose.
2. Seeds must fail before loops may pass.
3. Tools stay ON — the loop is the exam.
4. Single-shot is canonical; reruns are footnotes.
5. **A zero is the stack until proven otherwise.**
6. Attribute provider × model, always.
7. Timeout every call — and every grader.
8. Uniform failure across models = your bug.
9. Preserve broken runs; archive, never delete.
10. Verify the verifiers — oracles have bugs too.

KEEP GOING

Resources & next steps

The repo (everything today)

`github.com/rick-stevens-ai/pi-30`

problems · harness · runs · report · deck

pi itself

`pi.dev` · `pi.dev/docs/latest`

install, providers, extensions, skills, SDK

pi source & community

`github.com/earendil-works/pi` · `Discord`

MIT · issues welcome · session-sharing for OSS training

The written report

`report/FLEET_BENCHMARK_REPORT.pdf (v8)`

the master table + per-problem analysis, citable

Homework (pick one): ① finish your Lab 2 lane and read every verdict · ② run a second provider and diff the two lanes · ③ author P31 against the checklist and open a PR.

3:00 · THANK YOU

Go close some loops.

Questions — and live scores from the Lab 2 lanes still running.

Verdicts from exit codes, never vibes. — the whole tutorial, in one line

Rick Stevens · rick.stevens.ai@gmail.com · github.com/rick-stevens-ai/pi-30